



Embedded AI Narrative Device

Tutorial – Strategy 1 (Edge + Cloud Narrative)



Goal

In this tutorial, we build a system that senses simple events in a home, transforms them into structured data, sends them to a server, and turns them into a short narrative using AI. The final output is experienced as a “story of the day” told by a device that feels like a presence in the house.

This is important: we are not building a smart home controller. We are designing a narrative system. The technical pipeline exists to support meaning, not functionality.



SYSTEM OVERVIEW

At a high level, the system is composed of five stages. Each stage has a clear responsibility, and keeping them separate will make your project easier to build and evolve.

Sensors → Microcontroller → Events (JSON) → Server → AI → Output

Think of this as a translation pipeline:

- the physical world becomes signals
 - signals become events
 - events become language
-



Sensors – Reading the environment

What we do

Sensors convert physical phenomena into electrical signals. They do not “understand” anything, they simply measure changes.

This limitation is actually useful: by restricting what we sense, we force ourselves to design interpretation.

Recommended sensors

- PIR → detects motion (binary: movement or no movement)
- Light sensor (LDR) → detects light intensity (analog value)
- Microphone (analog) → detects sound level, not content

- Reed switch → detects door open/close

How to think about sensors

Each sensor is a **hint**, not a fact.

For example:

- motion detected $\neq$ a person is in the room
- light on $\neq$ someone is present

Your job is to combine hints into meaningful events.

2 Microcontroller – ESP32

What it is

A microcontroller is a small computer that runs simple programs and interacts with hardware.

We use ESP32 because:

- it has built-in WiFi
- it is inexpensive
- it is widely supported in teaching environments

What happens here

The microcontroller reads raw sensor values and converts them into **interpretable signals**.

Example:

- PIR HIGH → movement detected
- LDR value increases → light turned on

Important idea

This stage is about **simplification**, not intelligence.

We are not trying to “understand behavior” here, only to detect and label basic changes.

3 Events – Understanding JSON

What is an event?

An event is a structured description of something that happened.

Instead of writing vague text like:

```
motion detected
```

we create structured data that can be processed reliably.

What is JSON?

JSON (JavaScript Object Notation) is a simple format to represent data using key-value pairs.

Example:

```
{
  "time": "15:03",
  "location": "kitchen",
  "type": "movement",
  "duration": "short"
}
```

How to read JSON

- keys ("time", "location") describe what the data means
- values ("15:03", "kitchen") are the actual data

Why JSON matters

JSON is the bridge between hardware and software systems because:

- it is human-readable
- it is machine-readable
- it is easy to send over the internet

Designing good events is one of the most important parts of this project.

WiFi – Sending data

What happens

The ESP32 connects to a WiFi network and sends events to a remote server.

This is similar to how a browser sends data to a website.

Conceptual model

The device does not store complex knowledge. It **reports what it sees**.

Device → sends event → server receives

What students implement

- connect ESP32 to WiFi
- send JSON using HTTP requests

You do not need deep networking knowledge. Focus on the idea of communication.

5 Server – Organizing the system

What it is

A server is a program that receives data and decides what to do with it.

Typical options:

- Python with Flask
- Node.js

What it does

1. receives JSON events
2. optionally stores them
3. forwards them to an AI model

Why we need a server

The server acts as a **bridge** between hardware and AI services. It allows you to:

- preprocess data
 - add logic
 - control how AI is used
-

6 AI – Generating the narrative

Role of AI

The AI does not detect events. It transforms structured data into language.

This is where meaning, tone, and personality emerge.

Example input

```
{
  "events": [
    {"time": "15:03", "type": "movement", "location": "kitchen"},
    {"time": "18:20", "type": "door_open"}
  ],
  "personality": "gossiping roommate"
}
```

Example prompt

“Turn these events into a short narrative from the perspective of a curious housemate.”

Example output

“It was mostly quiet today, but someone rushed through the kitchen in the afternoon...”

Important idea

AI is used for:

- narrative generation
- interpretation style
- personality

Not for sensing or raw data processing.

Output – Bringing it back to the object

Why output matters

The output defines how the system is experienced. It transforms data into presence.

Options

A. Sound

- speaker
- MP3 module
- text-to-speech

B. Physical feedback

- LED patterns
- movement
- material response

Design insight

Avoid creating a “voice assistant”.

Instead, design an object that communicates through a combination of channels.

FULL PIPELINE EXPLAINED

```
Sensors
↓
Microcontroller reads signals
↓
Signals become structured events (JSON)
↓
Events are sent via WiFi
↓
Server processes and forwards
↓
AI generates narrative
↓
Device outputs story
```

Each step transforms information into a more meaningful form.

STEP-BY-STEP IMPLEMENTATION

Step 1 – Read a sensor

Start simple. Connect a PIR sensor and print its value.

Goal: understand hardware input.

Step 2 – Create an event

Replace raw messages with structured data.

```
{  
  "type": "movement"  
}
```

Goal: introduce abstraction.

Step 3 – Add context

Add time and location.

```
{  
  "time": "15:03",  
  "location": "kitchen",  
  "type": "movement"  
}
```

Goal: make events meaningful.

Step 4 – Send via WiFi

Send JSON to the server.

Goal: enable communication.

Step 5 – Build server

Receive and print events.

Goal: understand system flow.

Step 6 – Connect AI

Send events to AI and generate text.

Goal: transform data into narrative.

Step 7 – Output story

Play audio or display results.

Goal: complete the experience.



COMMON MISTAKES

- ✗ Using complex sensors too early (camera, speech recognition) → increases complexity without adding value
 - ✗ Trying to run AI locally on microcontroller → not necessary for this strategy
 - ✗ Mixing all logic in one place → makes system hard to debug
 - ✗ Ignoring data structure → prevents future evolution
-



KEY IDEA

The system does not understand reality.

It translates:

- signals → events
- events → narrative

Meaning is not detected. It is constructed.

This mindset is what turns a technical prototype into a design project.